PASS 0: ARCHITECTURE VALIDATION & HEALTH REPORT

Step 1: Load Order Foundation Analysis

Extract .toc information:

1. Read the .toc file
2. Extract load order (line numbers matter)
3. Identify "Core Foundation" section (loads first)
4. Extract SavedVariables declarations
5. List all files in load order

Validate foundational files:

```
Required first (in order):
- events.lua (or equivalent event system)
- eventsInternal.lua (internal event bus)
- dependencies.lua (module registration system)

Required before most modules:
- utils.lua (if used directly without lazy-loading)
- constants.lua
- data.lua (SavedVariable gateway)

Everything else: Order doesn't matter (dependency system handles it)
```

Step 2: Dependency System Analysis

Parse all module registrations:

```
// Search for patterns:
Addon.registerModule("moduleName", {"dep1", "dep2"}, function)
Addon.autoRegister("moduleName", {"dep1"}, moduleObject)

// Extract:
- Module name
- Declared dependencies array
- Whether it has initialize() function
```

Build dependency graph:

1. For each module, list its dependencies
2. Verify each dependency exists (search for `Addon.depName =`)
3. Build parent→child relationships
4. Detect circular dependencies (A→B→A chains)

Dependency validation:

```
✓ Module declares dependencies
✓ All dependencies exist in project
✓ No circular dependency chains
✗ Module depends on non-existent module
✗ Circular: A → B → C → A
```

Step 3: Module Reference Counting

For each module export:

1. Search entire project for `Addon.moduleName` pattern
2. Count occurrences (excluding self-export line)
3. Categorize by reference count:
    - 0 refs = DEAD CODE
    - 1 ref (self only) = ORPHANED
    - 2-5 refs = LOW USAGE
    - 6+ refs = ACTIVE

Generate reference report:

```
HIGH USAGE (core modules):
- Addon.utils: 147 references
- Addon.events: 89 references

ACTIVE:
- Addon.options: 23 references
- Addon.entityData: 15 references

LOW USAGE (investigate):
- Addon.exports: 3 references
- Addon.stateMachine: 2 references

ORPHANED (likely dead):
- Addon.modules.achievements: 1 reference (self only)
- Addon.modules.trading: 1 reference (self only)
```

Step 4: Dead Code Detection

Files not in .toc:

```
# List all .lua files in project
# Cross-reference against .toc
# Report files not loaded
```

Modules never called:

- Exports exist (`Addon.X = ...`)
- But no usage found in any file
- Not in any `registerModule` dependency array

Unused functions:

- Function defined but no callers found
- Exclude: `initialize()`, event handlers, registered callbacks

Step 5: SavedVariable Lifecycle Validation

Check for timing violations:

```
// BAD - File-load-time access:
local _, Addon = ...
if not pao_settings then  -- ✗ WRONG: SavedVariables not ready yet
    return {}
end

// GOOD - Initialize-time access:
function module:initialize()
    if not pao_settings then  -- ✓ CORRECT: After ADDON_LOADED
        pao_settings = {}
    end
end
```

Pattern to detect:

- SavedVariable reference outside any function
- SavedVariable check before module registration
- Direct SavedVariable access in module body (not in function)

Step 6: Generate Mermaid Dependency Graph

Create visual dependency graph:

```
graph TD
    %% Core foundation
    events[events]
    eventsInternal[eventsInternal]
    dependencies[dependencies]
    utils[utils]

    %% Module dependencies
    options[options] --&gt; utils
    petBattle[petBattle] --&gt; options
```

```
    petBattle --&gt; utils
    infoPanel[infoPanel] --&gt; options

    %% Orphaned modules (red)
    achievements[achievements] --&gt; utils

    %% Style by category
    style events fill:#90EE90
    style eventsInternal fill:#90EE90
    style dependencies fill:#90EE90
    style utils fill:#87CEEB
    style achievements fill:#FFB6B6
```

Graph features:

- Green: Foundational modules (pre-initialized)
- Blue: Core services (high usage)
- Yellow: Active modules (moderate usage)
- Red: Orphaned/dead modules
- Arrows show dependency direction

Step 7: Architectural Health Report

Output comprehensive report:

```
=== ARCHITECTURAL HEALTH REPORT ===

DEPENDENCY GRAPH: [Mermaid diagram above]

FOUNDATIONAL FILES (Load Order Critical):
✓ events.lua (line 10)
✓ eventsInternal.lua (line 11)
✓ dependencies.lua (line 13)
✓ utils.lua (line 19)

DEAD CODE (Delete):
☠️ core/npcDetection.lua - Not in .toc, depends on non-existent
Addon.npcDetector
☠️ core/npcData.lua - Not in .toc, zero references

ORPHANED MODULES (Investigate):
⚠️ modules/achievements.lua - Registered but only 1 reference (self)
⚠️ modules/collection.lua - Registered but only 1 reference (self)
⚠️ modules/leveling.lua - Registered but only 1 reference (self)
⚠️ modules/trading.lua - Registered but only 1 reference (self)

DEPENDENCY ISSUES:
🔴 CRITICAL: core/abilityData.lua references Addon.npcCommitter (doesn't
```

```
exist)
   Should be: Addon.entityCommitter

CIRCULAR DEPENDENCIES:
[none detected] or [list any A→B→A chains]

MODULE REFERENCE COUNTS:
Core (50+ refs): utils(147), events(89), constants(67)
Active (10-49): options(23), entityData(15), petUtils(12)
Low (2-9): exports(3), stateMachine(2)
Orphaned (1): achievements(1), trading(1)

SAVEDVARIABLE TIMING VIOLATIONS:
╳ options.lua:11 - Checks pao_settings at file-load time (should be in
initialize())

STATISTICS:
- Total modules: 69
- Dead code files: 2
- Orphaned modules: 4
- Missing dependencies: 1
- Circular dependencies: 0
- Timing violations: 1
```

PASS 1: SYNTAX & BASIC ERRORS

Check for:

- Undefined variables (excluding WoW API globals)
- Typos in module names (`Addon.npcDectector` vs `Addon.npcDetector`)
- Case sensitivity issues (`Addon.Data` vs `Addon.data`)
- Unreachable code (after return, after error())
- SavedVariable case must match .toc exactly
- Missing required parameters in function calls
- Bracket/parenthesis mismatches

WoW API globals to ignore:

- All C_* namespaces (C_PetJournal, C_PetBattles, C_Timer, etc.)
- UIParent, GameTooltip, CreateFrame, etc.
- WoW events (ADDON_LOADED, PET_BATTLE_OPENING_START, etc.)

PASS 2: LOGIC & FLOW

Check for:

- Nil safety issues (calling methods on potentially nil values)
- Loop conditions that never terminate or never execute
- Return statements missing in all code paths
- Off-by-one errors in loops
- Incorrect boolean logic
- Type mismatches (string vs number operations)

PASS 3: WOW-SPECIFIC PATTERNS

Event Registration Patterns

```
// CORRECT: Module registers own events in initialize()
function module:initialize()
    Addon.events:registerEvent("EVENT_NAME", function(event, ...)
        -- handler
    end)
end

// WRONG: Events registered in file body (before events system ready)
Addon.events:registerEvent("EVENT_NAME", ...)  -- At top level
```

SavedVariable Patterns

```
// CORRECT: Check in initialize()
function module:initialize()
    if not pao_settings then
        pao_settings = {}
    end
end

// WRONG: Check at file load
if not pao_settings then  -- At top level
    return {}
end
```

Lazy-Loading Patterns (Foundational files only)

```
// For files in "Core Foundation" section:
local utils = nil  -- Declare as nil

function someFunction()
    if not utils then utils = Addon.utils end
    utils:debug("message")
end
```

```
// Or assign in initialize():
local utils = nil
function module:initialize()
    utils = Addon.utils
end
```

PASS 4: PERFORMANCE PATTERNS

Check for:

Frame Creation

```
// BAD: CreateFrame in loop
for i = 1, 100 do
    local frame = CreateFrame("Frame")  -- ✗ Memory Leak
end

// GOOD: Pre-create frames, reuse
local frames = {}
for i = 1, 100 do
    frames[i] = CreateFrame("Frame")
end
```

String Concatenation

```
// BAD: .. in loop
local str = ""
for i = 1, 1000 do
    str = str .. data[i]  -- ✗ O(n²) performance
end

// GOOD: table.concat
local parts = {}
for i = 1, 1000 do
    parts[i] = data[i]
end
local str = table.concat(parts)
```

API Call Caching

```
// BAD: Repeated expensive calls
for i = 1, numPets do
```

```
    local total = C_PetJournal.GetNumPets()  -- ✗ Called numPets times
    -- use total
end

// GOOD: Cache result
local total = C_PetJournal.GetNumPets()
for i = 1, total do
    -- use total
end
```

PASS 5: STANDARDS COMPLIANCE

Based on project PERSONA rules:

Naming Conventions

- Variables/Functions: lowerCamelCase
- Methods: Colon syntax (`:method()`)
- Constants: CONSTANT_NAME
- Globals: pao_ prefix
- Module exports: Addon.moduleName

Documentation

```
// GOOD: Complete documentation
--[[
  Description of function
  @param paramName type - Description
  @return type - Description
]]
function module:functionName(paramName)

// BAD: Missing @return
function module:functionName(paramName)
    return value  -- What does it return?
end
```

No Placeholders

```
// BAD:
function todo()
    -- TODO: Implement this
end
```

```
// BAD:
if condition then
    -- Placeholder logic
    return nil
end

// GOOD: Complete, functional code only
```

Separation of Concerns

```
// Check file paths and responsibilities:
ui/*.lua - Frames, widgets, visual layout ONLY
logic/*.lua - Business rules, calculations ONLY
data/*.lua - Static tables, SavedVariables ONLY
core/*.lua - Cross-cutting utilities ONLY

// BAD: UI code in logic file
function logic:calculateDamage()
    local frame = CreateFrame("Frame")  -- ✗ UI in logic
    return damage
end
```

OUTPUT FORMAT

Report Structure:

```
=== PASS 0: ARCHITECTURAL HEALTH REPORT ===
[Mermaid dependency graph]
[Dead code, orphans, missing deps, circular deps]
[Reference counts, timing violations]

=== PASS 1-5: CODE QUALITY ISSUES ===

CRITICAL (Fix immediately):
- file.lua:line - Exact issue description
  Recommended fix: [specific solution with code]

HIGH:
- file.lua:line - Issue description
  Recommended fix: [specific solution]

MEDIUM:
- file.lua:line - Issue description

LOW:
```

```
- file.lua:line - Issue description

=== STATISTICS ===
- Total Issues: X (Critical: X, High: X, Medium: X, Low: X)
- Files Checked: X
- Dead Code Files: X
- Orphaned Modules: X
- Files with Zero Issues: X
- Recommended Deletions: X
```

Issue Format:

- File:Line - Brief description (1 line)
- Why it matters - Impact (1-2 sentences)
- Recommended fix - Specific solution with code example (Critical/High only)

Caps:

- Maximum 25 content issues (Passes 1-5)
- Unlimited architectural issues (Pass 0)
- Provide inline code fixes for Critical/High only

SPECIAL CASES

When user says "lint core folder":

1. Run Pass 0 on entire project (architecture needs full view)
2. Run Passes 1-5 only on core/*.lua files
3. Generate health report showing core's role in architecture

When project knowledge is stale:

- Always ask user to upload files being linted
- Use uploaded files as source of truth
- Project knowledge = reference only

When to stop:

- After ~25 content issues found
- Architecture report is always complete
- Prioritize worst offenders